

8051 - Microcontroller

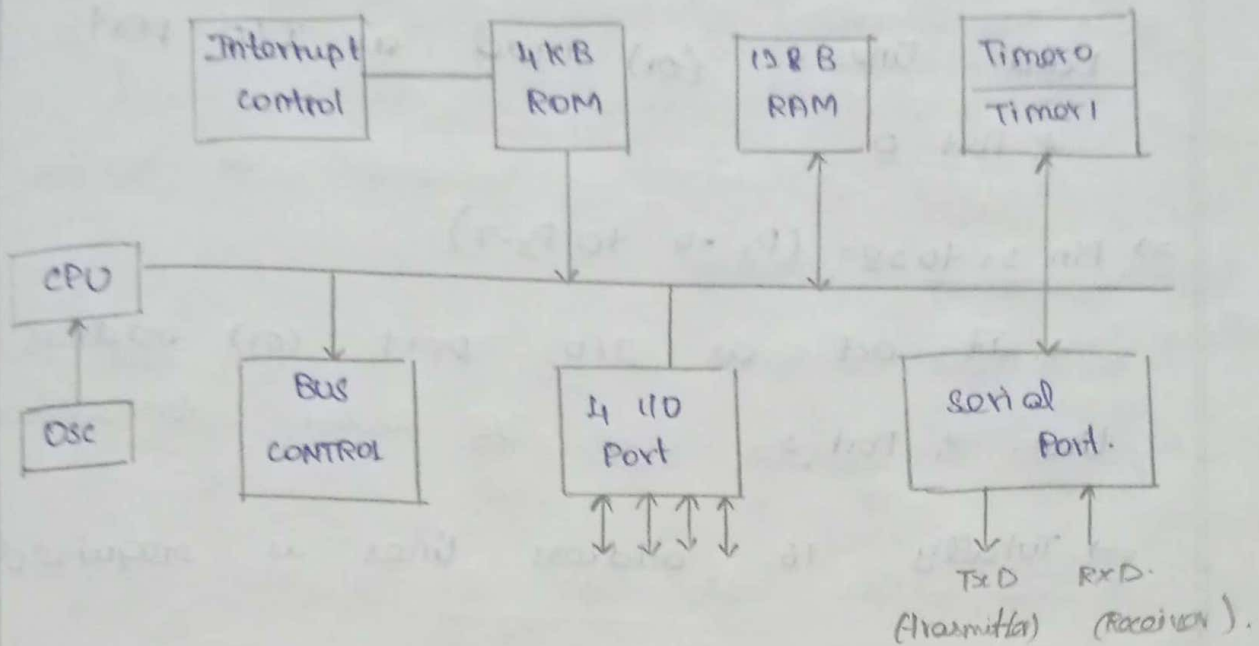
Microprocessor	Microcontroller.
* CPU stand alone, IO, timer, UART, memory separate * used for general purpose application.	* Everything on the same chip. * used for single application.

⇒ It is a 8-bit microcontroller.

Features:

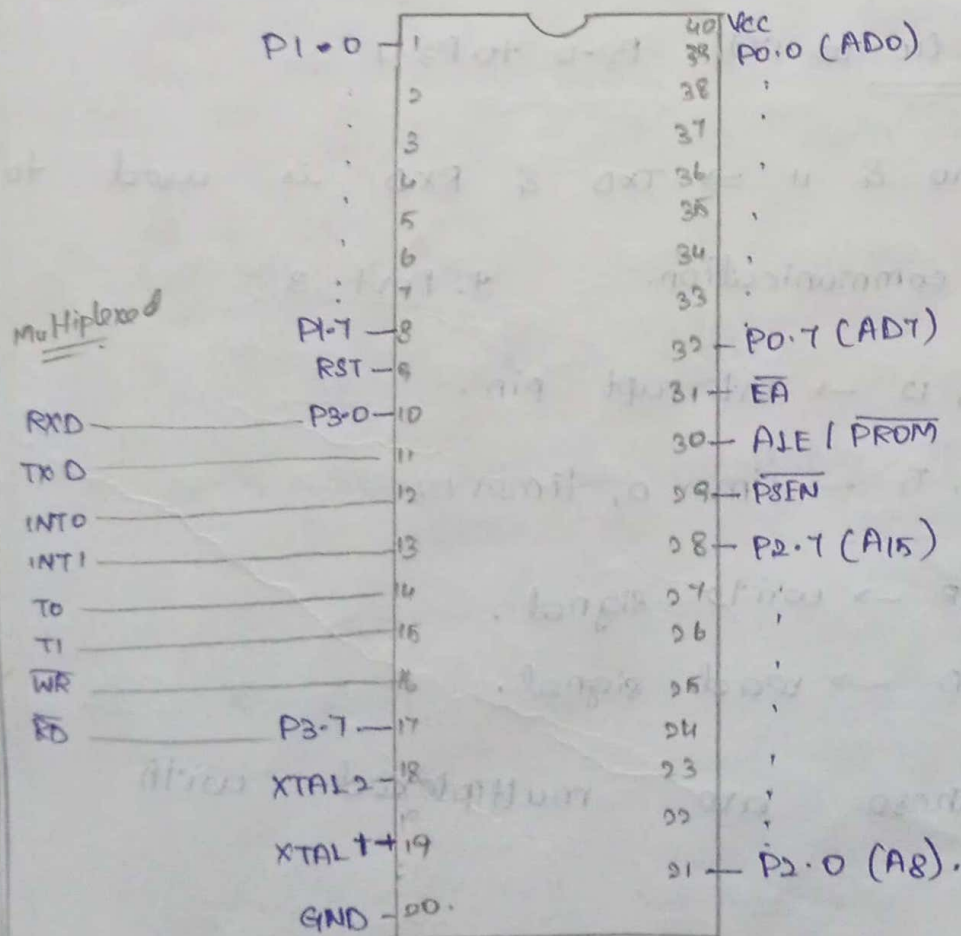
- ① It is 8-bit.
- ② It contains 64 kbyte Program memory space and 64 kbyte data memory address space.
- ③ On chip 4 kilobyte program memory contains 128 byte data memory.
- ④ Contains two 16 bit timers.
- ⑤ Contains UART for serial communication.
- ⑥ 4 I/O ports, each port has 8 lines, totally there will be 32 I/O lines.

Architecture diagram of 8051.



Pin diagram of 8051:

⇒ 40 pin package:



⇒ Pin 39 to 32: (AD₀-AD₁)

* It is used as multiplexed address & Data lines (or) used as I/O port.
* Port 0.

⇒ Pin 21 to 28: (P₂-0 to P₂-7)

* It act as I/O port (or) address line.
* Port 2 - used as higher order address.

⇒ Totally 16 address lines is required.

⇒ Pin (1 to 8):

* Port 1 ⇒ used as I/O port.

⇒ Pin (10 to 17): P₃-0 to P₃-7

* Pin 10 & 11 ⇒ TXD & RXD is used to serial communication.
* Port 3.

* 11, 12 → Interrupt pin.

* T₀, T₁ → timer 0, timer 1.

* $\overline{WR}$ → write signal.

* $\overline{RD}$ → read signal.

• These are multiplexed with Port - 3.

⇒ Pin 18:

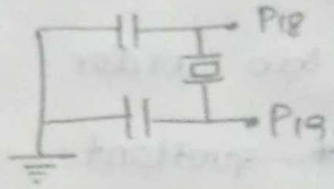
* XTAL 2

⇒ Pin 19:

* XTAL 1.

⇒ crystal oscillator is connected btw 18 & 19 pins.

⇒ If we use TTL oscillator directly connect to XTAL1.



⇒ Pin 31: $\overline{EA}$

* $\overline{EA}$ is connect to V_{cc} . (when external memory not needed)
 $\overline{EA}$ - External access.

* when program is in external memory, $\overline{EA}$ is connected to ground.

⇒ Pin 30: ALE / $\overline{PROM}$

* Address latch enable.

if, 1 → act as address line.
0 → act as data line.

⇒ Pin 29: $\overline{PSEN}$

* Program store enable

if program is available in external address } → 0.

if data available in external memory } → 1.

Registers
* ST has A & B register - 8 bit each.

A → low order 8 bit. (for all arithmetic work)

* quotient.

* one operand is stored and result is stored.

B → high order of 8 bit. (for xly & + we have to use this).

* Remainder stored.

* we have R0 to R7 registers (8 bit).

* stack pointer SP (8 bit)

→ for every push operation, it is incremented by 1.

→ for pop operation, it is decrement by 1.

SP → defines the top of stack

* when stack is empty, SP has 07.

* Program counter (16 bit) - PC

→ used to store address of next instruction.

* Program status word register - PSW.

AC → auxiliary carry.

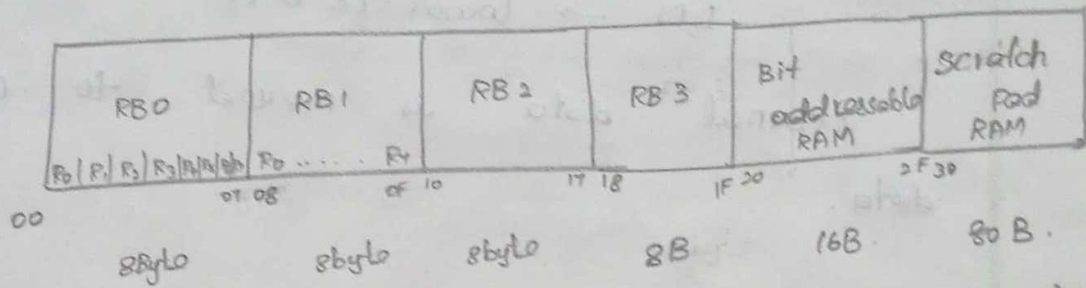
FO → for general purpose.

RSI, RSO → Register select the register bank.

CY → carry.

CY	AC	FO	RS1	RS0	OV	-	P.
----	----	----	-----	-----	----	---	----

128 bytes separation:



⇒ 32 byte of memory in Register bank (RB).

eg: MOV A, R0. ⇒ Move data in R0 to A

⇒ If RS1 RS0 is 00 then.

data of R0 in Bank 0 is moved to Accumulator.

RS1 RS0

0 0 → Bank 0

0 1 → Bank 1

1 0 → Bank 2

1 1 → Bank 3.

⇒ each & every bit can be accessed in the address space - Bit addressable RAM.
 we can read a particular bit.
 ⇒ Scratch Pad RAM.

* for read / write operation storage we can use this space.

⇒. totally 128 bytes.

RB0	-	8B	} Byte addressable RAM.
RB1	-	8B	
RB2	-	8B	
RB3	-	8B	
Bit address RAM	-	16B	
Scratch Pad RAM	-	80B.	

128B

* Data point register (16 bit) DPTR,

DPH $\rightarrow$ higher (8 bit)

DPL $\rightarrow$ lower (8 bit).

External data is moved to controller data.

ADDRESSING MODES:

1. Immediate addressing mode:

* operand is given in the instruction itself directly.

* without # it is considered as address. with # it is data.

eg: MOV A, #45H

ADD A, #44H.

2. Register Addressing mode:

* If the data source given in the Register.

eg: MOV A, B.

MOV A, R₁.

MOV B, A

MOV R₀, R₁ [Not possible]

↓ it is done by

(MOV A, R₁

MOV R₀, A)

3). Direct Addressing Mode:

* If the address of the source operand is given in the instruction, it is called direct addressing mode.

eg: MOV A, 10H.

4). Register Indirect Mode:

* The address of source operand is given in the Register, it is called Register Indirect Mode.

eg: MOVX A, @DPTR;

5). Indexed Addressing Mode:

eg: MOVX A, @DPTR + A.

@DPTR 1400

1400 address 05

A 03

@DPTR + A 07 → is moved to A 07

INSTRUCTION SETS:

①. Data transfer Instruction:

* MOV — transfer a data from source to destination.

eg: MOV A, B.

* MOVC — used to move a code byte from ^{internal} memory to accumulator.

eg: MOVC A, @R0.

[@ - act as a pointer to the internal memory]

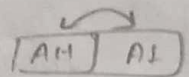
* MOVX — to move a byte from/to external memory to accumulator.

eg: MOVX A, @DPTR.

MOVX @DPTR, A.

* SWAP — swap the content of the accumulator upper & lower nibble.
4 byte.

eg: SWAP A.



* XCH — Exchange the content of two Register.

eg: XCH A, R1

* PUSH — to move a content to the stack.

eg: PUSH DPTR, PUSH DPL

③ Logical Instruction:

*. ANL - AND operation.

eg: ANL A, B

*. XRL - XOR operation.

eg: XRL A, B

*. ORL - OR operation

eg: ORL A, B

*. CPL - Complement

eg: CPL A

2's complement of A

*. CLR - clear.

eg: CLR A

data of A is Reset to '0'

*. RR - Rotate Right.

LSB $\rightarrow$ MSB.

eg: RR A, [1 time]

RR A, 02 [2 times]

*. RRC - Rotate Right through carry.

LSB $\rightarrow$ carry $\rightarrow$ MSB.

*. RL - Rotate Left.

eg: RL A

*. RLC - Rotate Left through carry.

MSB $\rightarrow$ carry $\rightarrow$ LSB.

④. Bit - Manipulation Instructions:

* CLR - clear instruction.

* SETB - set bit

* JbC - Jump carry, [base on carry bit]

* JNC - Jump if no carry.

⑤. Branching Instruction.

* LJMP - Long jump. (3 byte instruction)
 the instruction to complete, it takes 3 cycle

* AJMP - Absolute jump. (2 byte instruction)
 11 - address lines.

* SJMP - short jump.

* LCALL - long call

* ACALL - absolute call. (2 byte)

* JN - jump if zero.

* JNZ - jump if not zero.

* CJNE - Compare jump if not equal.

* DJNZ - Decrement jump if not zero.

* RET - Return.

* RETI - Return Interrupt.

(in interrupt we use)

eg: CJNE R₂, #05H, address.

if R₂ ≠ #05H, it will branch to that address.

⇒ It will branch into particular bit

if zero flag is 1.

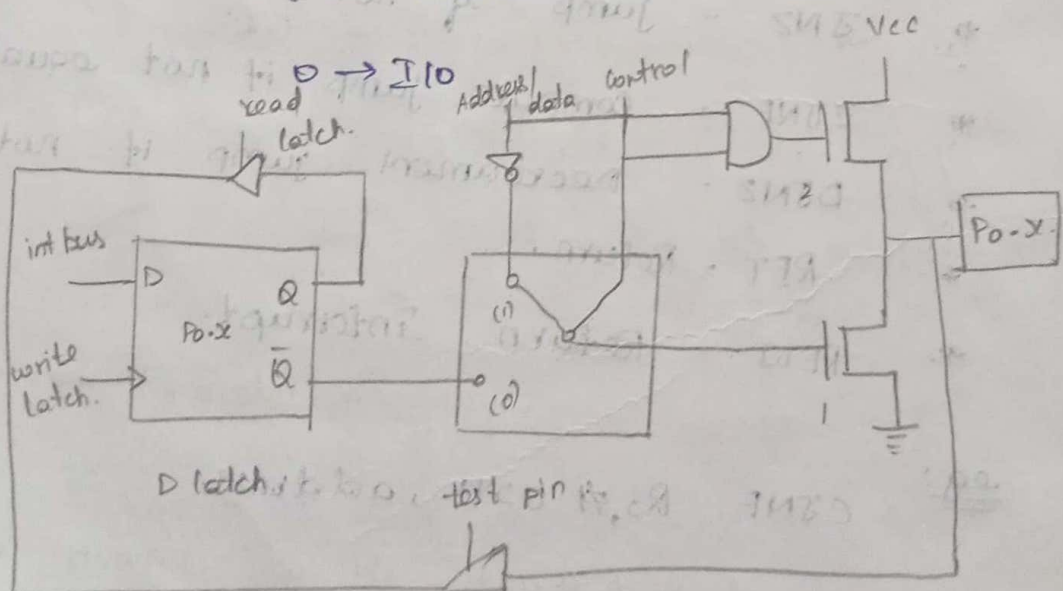
eg: DJNZ R₂, address.

if R₂ ≠ 0, it will branch to address.

210 Pins, Ports and circuits.

It is based on control word.

1 → address | data | no.



→ assume, control = 0.

* Switch connected to o/p of $\bar{Q}$

(used as simple i/o)

→ I want to use i/p port.

* write 1 to latch.

$Q \rightarrow 1$

$\bar{Q} \rightarrow 0$

switch $\rightarrow \bar{Q}$
connect.

→ when i/p $\rightarrow 0$ NMOS $\rightarrow$ OFF, CMOS $\rightarrow$ ON
i/p $\rightarrow 1$ NMOS $\rightarrow$ ON, CMOS $\rightarrow$ OFF.

both NMOS = 0, Port 0 of x^{th} pin is floating, by using external source it uses.

$\Rightarrow$ It used as i/p port.

for any control i/p = 0.

→ I want to use as o/p port.

latch $\rightarrow 1$.

$Q = 1$

$\bar{Q} = 0$

both NMOS = OFF

Port x is floating.

to get o/p = 1, we use pull resistor internally.

if latch $\rightarrow 0$.

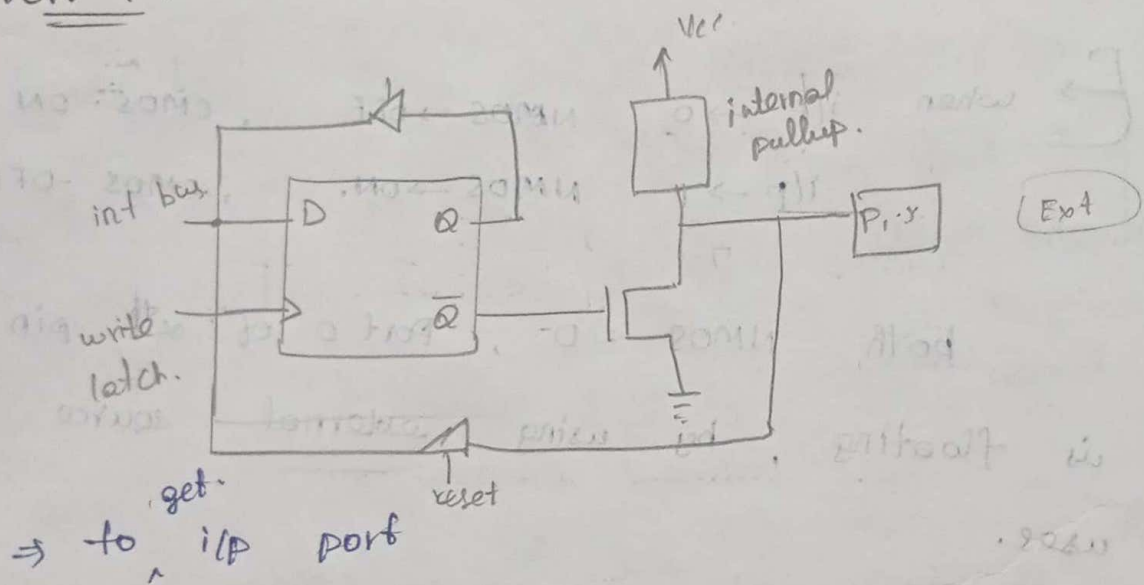
$$Q = 0$$

$$\bar{Q} = 1$$

① NMOS = ON

$\rightarrow$ Control $i/p = 1$ [address (data line)].

Port 1:



* write latch 1.

$$Q = 1$$

$$\bar{Q} = 0$$

$\rightarrow$ NMOS - OFF.

(eliminated)

* Through Int. pull up the Vcc path

go to P1.x.

* External source have the capability

to sink the current

4. write latch 0.

$Q = 0$

$\rightarrow$ NMOS - ON

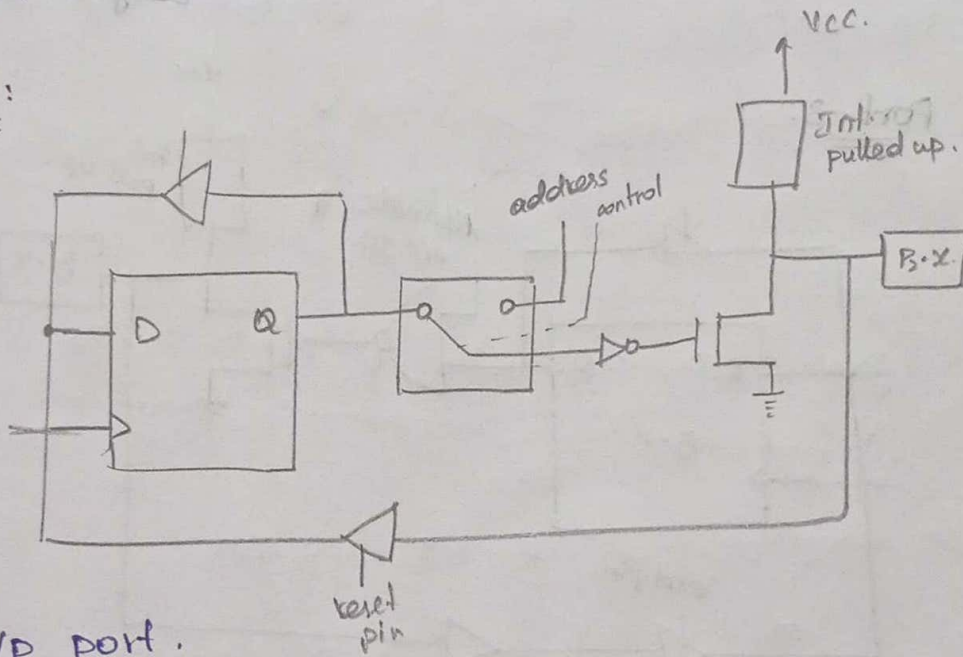
$\bar{Q} = 1$

(Connects ground)

we get '0'.

$\Rightarrow$ to get o/p port we won't use external source.

Port 2:



$\Rightarrow$ i/p port.

write latch 1.

control signal control's the switch.

control $\rightarrow 0$.

$Q = 1 \rightarrow D \rightarrow 0$. NMOS - OFF.

Using Ext source, write i/p to Port 2.X.

External source:

(It should be able to drain the current of Int. pull up)

$\Rightarrow$ o/p port.

follow same as i/p port without Ext. source.

$\Rightarrow$ address - 1

$Do \rightarrow 1$ NMOS - ON. (Ground).

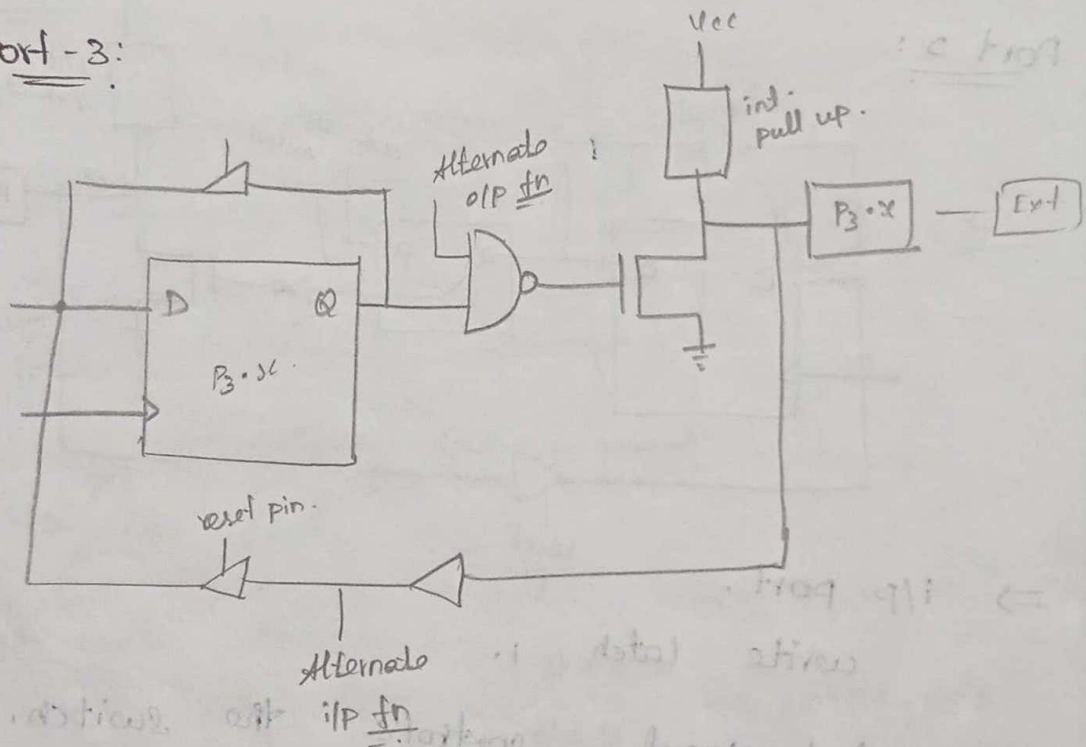
V_{cc} is connected, we get '1'

$\Rightarrow$ address - 0

$Do \rightarrow 0$ NMOS - OFF (connect $P_3.x$)

we get '0'.

Port - 3:



$\Rightarrow$ i/p $\rightarrow 1$

$Q \rightarrow 1$

$Do \rightarrow 0$ NMOS - OFF, $P_3.x$ only have
(have ext source)
give i/p.

$\Rightarrow$ o/p port

$\rightarrow 1$

$Q \rightarrow 1$

$Do \rightarrow 0$

NMOS - OFF, $P_3.x$ connect to V_{cc} .

(won't use ext. source)

If we set AH: 0/p $\frac{1}{2}$ we get $\rightarrow 1$ we get $\rightarrow 1$

we get $\rightarrow 0$, we get $\rightarrow 0$.

special Register:

That only used for particular to
not for general purpose.

eg: accumulator - to have the result of
Arithmetic & logic operation.